# Step 2c — Replay e oblio: perché qui (quasi) non servono

## Il punto

Nel piano generale il replay (../fine_tuning/02c_replay_e_forgetting.md) serviva a far restare il modello un decision model capace di rispondere a domande qualsiasi (`choice`, `noul`, `score`) dopo il fine-tuning. Con un compito solo binario **le altre domande non servono**: se il modello le dimentica, non si perde nulla di ciò che si usa.

Quindi:

- **niente replay obbligatorio**, niente generazione di domande generiche, niente target del teacher da calcolare;
- **niente blocco "oblio"** nella valutazione (JevBench, Kev-v9, typed-decisions);
- il training costa ~10–15% in meno di esempi e non serve il codice per gli esempi a più di due opzioni.

## Che cosa si rischia comunque di perdere

L'oblio che conta qui non è quello delle altre domande, ma quello **dentro il compito stesso**:

| Rischio | Come si manifesta | Perché succede |
|---|---|---|
| Calibrazione | probabilità tutte vicine a 0 o 1; soglia ottimale che scivola; ECE in aumento | la CE spinge senza limite sugli esempi già giusti; documenti sintetici molto "puliti" |
| Generalizzazione fuori dominio | ottimo su test_id, peggio della baseline su test_ood | il modello impara lo stile dei dati sintetici (MiniCheck lo combatte con il secondo stadio D2C) |
| Robustezza del prompt | crollo se cambia una virgola del prompt | solo se il prompt in produzione non è identico a quello di training (02d) |

## Gli strumenti che restano (opzionali)

### 1. Ancoraggio al modello di partenza sul compito stesso

Invece del replay su domande generiche, un termine di KL **sugli esempi di fact checking** verso il modello di partenza congelato:

```
L = CE(lettere, etichetta) + β · KL(p_partenza ‖ p_studente)      con β ∈ {0,05; 0,1}
```

Effetto: dove il modello di partenza era già giusto il training non lo sposta inutilmente; dove sbagliava, la CE prevale. Costo: un forward in più del modello di partenza per esempio, oppure le sue probabilità calcolate una volta prima del training (consigliato: è una sola inferenza sul mix).

Nota: con `support_simple` i decision model tendono a dire "sì" troppo facilmente (soglia ottimale alta, 01). Un β alto **conserverebbe** anche questo difetto. Per questo β resta piccolo, e l'ancoraggio è un esperimento (P-KL in 07), non il default.

### 2. Replay minimo per conservare il decision model (solo se serve)

Se si vuole che lo stesso modello resti utilizzabile anche per altre domande (per esempio un sistema che già lo usa per altro), si torna al replay del piano generale con quota bassa (5–10%). In quel caso questo piano e quello generale convergono: meglio seguire direttamente ../fine_tuning/.

**3. Due stadi invece del replay**

Il curriculum di MiniCheck e FactCG (02e) è già una forma di difesa: il secondo stadio, su dati realistici e a learning rate basso, "riporta" il modello verso la distribuzione d'uso dopo il primo stadio su dati sintetici.

## Cosa misurare al posto dell'oblio

| Misura | Su cosa | Allarme |
|---|---|---|
| ECE e Brier | `dev_select` | ECE sopra la baseline |
| Distanza fra BAcc a soglia 0,5 e a soglia fittata | `dev_select` | > 1 punto |
| BAcc su test_ood (alla fine) | test_ood | sotto la baseline |
| (opzionale) una domanda non di fact checking | 200 domande di typed-decisions test | solo per sapere quanto il modello si è specializzato; non è un vincolo |

## Criteri per passare allo step successivo

- Deciso se il modello deve restare anche decision model generale (→ piano generale) o no (→ questo piano, senza replay).
- Se si userà P-KL: probabilità del modello di partenza calcolate sul mix e salvate accanto alle righe.